Embedded System Engineer

We are building a wearable AI assistant.

It clips to a shirt, listens for a command, and talks to an LLM (Large Language Model) in the cloud.

The Challenge:
It must run on a tiny battery for **12 hours**.

Part 1: Hardware & Architecture

Goal: Prove you understand low-power design and audio processing.

Question 1: The "Always-Listening" Problem

To catch a wake word (like *"Hey Computer"*), the microphone must always be on.
This drains the battery.

Proposed Design: Using a dual-core SoC (System on Chip)

- **Core A:**
  - High performance
  - Wi-Fi capable
  - Power hungry
- **Core B:**
  - Low power
  - Slow speed
  - Limited RAM

Your Task:
Explain how you would split the work between Core A and Core B to maximize battery life.

- Which core handles wake-word detection?
- Which core handles cloud upload?

Question 2: Memory Calculation (Audio Buffer)

We need to record user audio before sending it to the cloud.

- **Format:**
  - 16 kHz sample rate
  - 16-bit depth (2 bytes per sample)
  - Mono
- **Duration:** 10 seconds
- **Constraint:** The chip has **256 KB of SRAM** available for this buffer.

Task:
Calculate the total size of the 10-second audio clip.

- Will it fit in RAM?
- Show your math.

Question 3: Privacy Hardware

Users are worried about being recorded secretly.

- We want a **"Hard Mute"** switch that physically cuts microphone power.

Schematic Check:
If the software crashes, can it accidentally turn the mic back on?

- How would you wire the switch to ensure *absolute* privacy?

Part 2: Firmware Logic (C Code)

Goal: Write a robust state machine for the device.

The Scenario

Write the C code that manages the device's main states.
You are writing the *"Brain"* of the pin.

Mock Functions (Assume these exist)

Mock Functions (Assume these exist)

- bool is_button_pressed(void)
  Returns true if the user holds the touch bar.
- int get_battery_level(void)
  Returns a value from 0 to 100.
- void set_led_color(int color)
  Colors: OFF, GREEN (Listening), RED (Error).
- void start_audio_stream(void)
  Starts sending audio to Wi-Fi.
- void stop_audio_stream(void)
  Stops sending audio.

The Logic Rules

1. **IDLE State**
   - Default state
   - LED is OFF
   - Audio is OFF
2. **LISTENING State**
   - Enter ONLY if:

        - Button is pressed **AND**
        - Battery > 10%
    - If battery < 10%, go to **ERROR** immediately
    - Turn LED GREEN
    - Start Audio
3. **Hold-to-Talk**
    - Device stays in LISTENING only while the button is held
4. **Finishing**
    - When the button is released:
        - Stop Audio
        - Turn LED OFF
        - Return to IDLE
5. **Safety**
    - If the button is stuck (held for > 30 seconds):
        - Force a reset to IDLE
        - Prevent overheating

Constraints

- Use a switch statement for the state machine
- Do **not** use floating point numbers
- Write clean, readable code